Use Matlab Function pwelch to Find Power Spectral Density – or Do It Yourself

In my last post, we saw that finding the spectrum of a signal requires several steps beyond computing the discrete Fourier transform (DFT) [1]. These include windowing the signal, taking the magnitude-squared of the DFT, and computing the vector of frequencies. The Matlab function `pwelch` [2] performs all these steps, and it also has the option to use DFT averaging to compute the so-called Welch power spectral density estimate [3,4].

In this article, I'll present some examples to show how to use pwelch. You can also "do it yourself", i.e. compute spectra using the Matlab fft or other fft function. As examples, the appendix provides two demonstration mfiles; one computes the spectrum without DFT averaging, and the other computes the spectrum with DFT averaging.

We'll use this form of the function call for pwelch:

```
[pxx,f]= pwelch(x,window,noverlap,nfft,fs)
```

The input arguments are:

x	input signal vector
window	window vector of length nfft
noverlap	number of overlapped samples (used for DFT averaging)
nfft	number of points in DFT
fs	sample rate in Hz

For this article, we'll assume x is real. Let length(x) = N. If DFT averaging is *not* desired, you set nfft equal to N and pwelch takes a single DFT of the windowed vector x. If DFT averaging is desired, you set nfft to a fraction of N and pwelch uses windowed segments of x of length nfft as inputs to the DFT. The $|X(k)|^2$ of the multiple DFT's are then averaged.

The output arguments are:

pxx	power spectral density vector, W/Hz
f	vector of frequency values from 0 to fs/2, Hz

The length of the output vectors is nfft/2 + 1 when nfft is even. For example, if nfft= 1024, pxx and f contain 513 samples. pxx has units of W/Hz when x has units of volts and load resistance is one ohm.

Examples

The following examples use a signal x consisting of a sine wave plus Gaussian noise. Here is the Matlab code to generate x, where sine frequency is 500 Hz and sample rate is 4000 Hz:

```
fs= 4000;                % Hz sample rate
Ts= 1/fs;
f0= 500;                 % Hz sine frequency
A= sqrt(2);              % V sine amplitude for P= 1 W into 1 ohm.

N= 1024;                 % number of time samples
n= 0:N-1;                % time index

x= A*sin(2*pi*f0*n*Ts) + .1*randn(1,N);    % 1 W sinewave + noise
```

The power of the sine wave into a 1-ohm load is $A^2/2 = 1$ watt. In the following, examples 1 and 2 compute a single DFT, while example 3 computes multiple DFT's and averages them. Example 4 calculates C/N_0 for the spectrum of Example 3.

Example 1. Spectrum in dBW/Hz

We'll start out with a rectangular window. Note the sinewave frequency of 500 Hz is at the center of a DFT bin to prevent spectral leakage [5]. Here is the code to compute the spectrum. Since we are not using DFT averaging, we set noverlap= 0:

```
nfft= N;
window= rectwin(nfft);
[pxx,f]= pwelch(x,window,0,nfft,fs);    % W/Hz  power spectral density

PdB_Hz= 10*log10(pxx);                  % dBW/Hz
```

The resulting spectrum is plotted in Figure 1, which shows the component at 500 Hz with amplitude of roughly -6 dBW/Hz. How does this relate to the actual power of the sinewave for a 1-ohm load resistance? To compute the power of the spectral component at 500 Hz, we need to convert the amplitude in dBW/Hz to dBW/bin as follows. First, convert W/Hz to W/bin:

Pbin (W/bin) = pxx (W/Hz) * fs/nfft (Hz/bin) (1)

Then, we have in dB:

PdB_bin = $10*\log_{10}$(pxx*fs/nfft) dBW/bin (2)

Or PdB_bin = PdB_Hz + $10*\log_{10}$(fs/nfft) dBW/bin

If we set the noise amplitude to zero, we get PdB_Hz = -5.9176 dB/Hz and

PdB_bin = -5.9176 dBW/Hz +10log$_{10}$(4000/1024) = 0 dBW = 1 watt

So we are reassured that the amplitude of the spectrum is correct. This motivates us to try the next example, which displays the spectrum in dBW/bin.

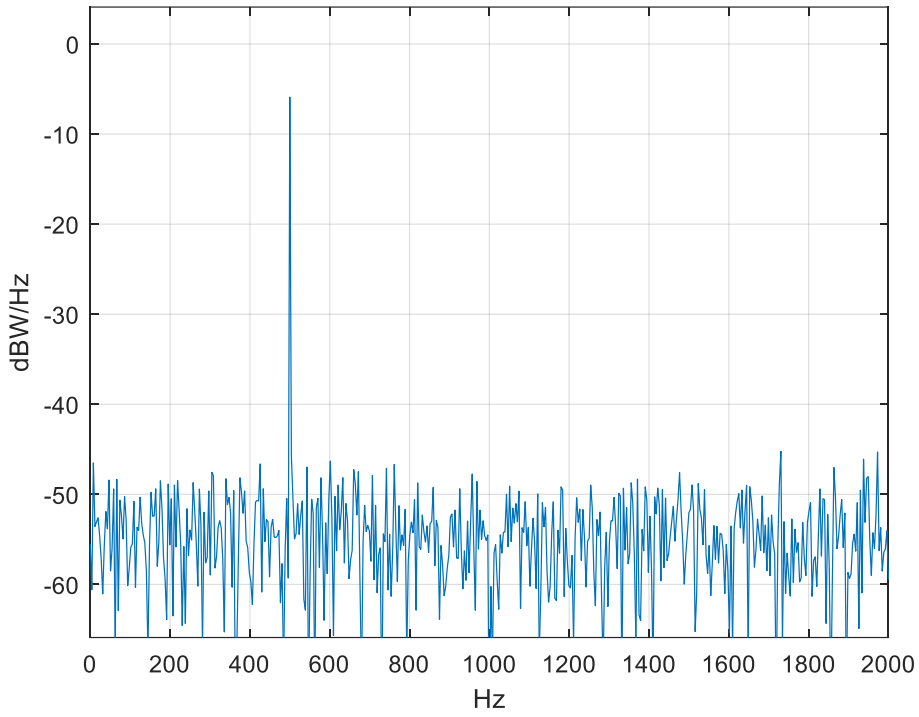


Figure 1. Spectrum in dBW/Hz

Example 2. Spectrum in dBW/bin

To find the spectrum in dBW/bin, the code is the same as Example 1, except we use Equation 2 to find PdB_bin:

```
nfft= N;
window= rectwin(nfft);
[pxx,f]= pwelch(x,window,0,nfft,fs);    % W/Hz  power spectral density

PdB_bin= 10*log10(pxx*fs/nfft);        % dBW/bin
```

The spectrum is shown in the top plot of Figure 2. As expected, the component at 500 Hz has power of 0 dBW. Note that the amplitude of a sine's spectral component in dBW/bin is constant vs nfft, while the amplitude in dBW/Hz varies vs. nfft. If we now replace the rectangular window with a hanning window:

```
window= hanning(nfft);
```

We get the spectrum in the bottom plot of Figure 2. The peak power is now less than 0 dBW, because the power is spread over several frequency samples of the window. Nevertheless, this is a more useful window than the rectangular window, which has bad spectral leakage when f_0 is not at the center of a bin.

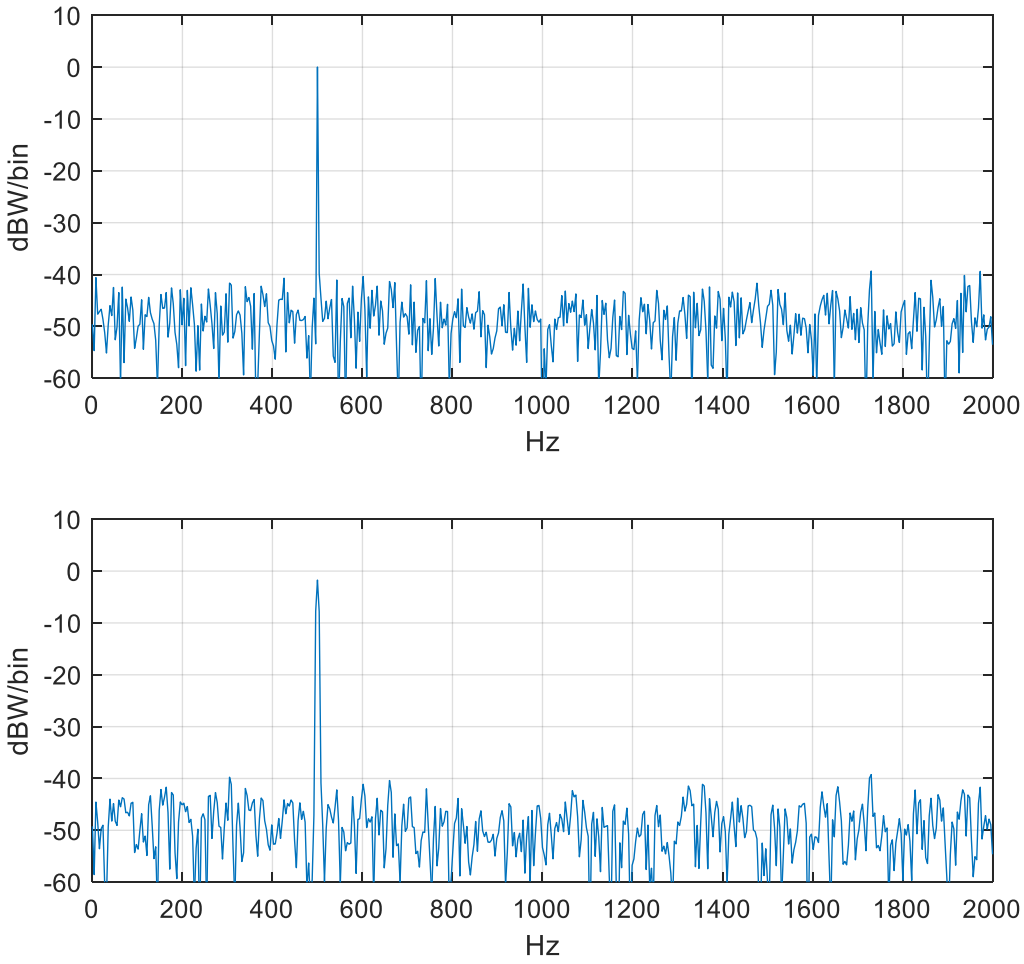


Figure 2. Spectra in dBW/bin Top: rectangular window
Bottom: hanning window

Example 3. DFT averaging

For this example, fs, Ts, f0, A, and nfft are the same as for the previous examples, but we use N= 8*nfft time samples of x. pwelch takes the DFT of N_{avg} = 15 overlapping segments of x, each of length nfft, then averages the $|X(k)|^2$ of the DFT's. The segments of x are shown in Figure 3, where we have set the number of overlapping samples to nfft/2 = 512.

In general, if there are an integer number of segments that cover all samples of N, we have [4]:

$$(N_{avg} - 1)*D + nfft = N \qquad (3)$$

Where D = nfft – noverlap. For our case, with D = nfft/2 and N/nfft = 8, we have

$$N_{avg} = 2*N/nfft - 1 = 15.$$

For a given number of time samples N, using overlapping segments lets us increase N_{avg} compared with no overlapping. In this case, overlapping of 50% increases N_{avg} from 8 to 15. Here is the Matlab code to compute the spectrum:

```
nfft= 1024;
N= nfft*8;                % number of samples in signal

n= 0:N-1;

x= A*sin(2*pi*f0*n*Ts) + .1*randn(1,N);    % 1 W sinewave + noise

noverlap= nfft/2;              % number of overlapping time samples
window= hanning(nfft);
[pxx,f]= pwelch(x,window,noverlap,nfft,fs);    % W/Hz  PSD

PdB_bin= 10*log10(pxx*fs/nfft);      % dBW/bin
```

The resulting spectrum is shown in Figure 4. Compared to the no-averaging case in Figure 2, there is less variation in the noise floor. DFT averaging reduces the variance σ^2 of the noise spectrum by a factor of N_{avg}, as long as noverlap is not greater than nfft/2 [6].

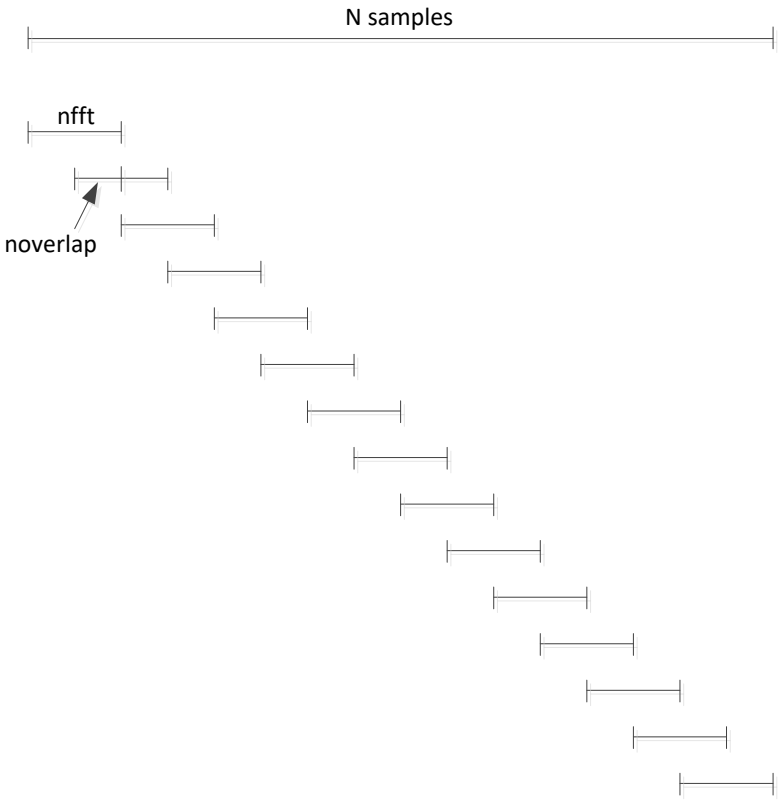


Figure 3. Overlapping segments of signal x for N = 8*nfft and noverlap = nfft/2 (50%).

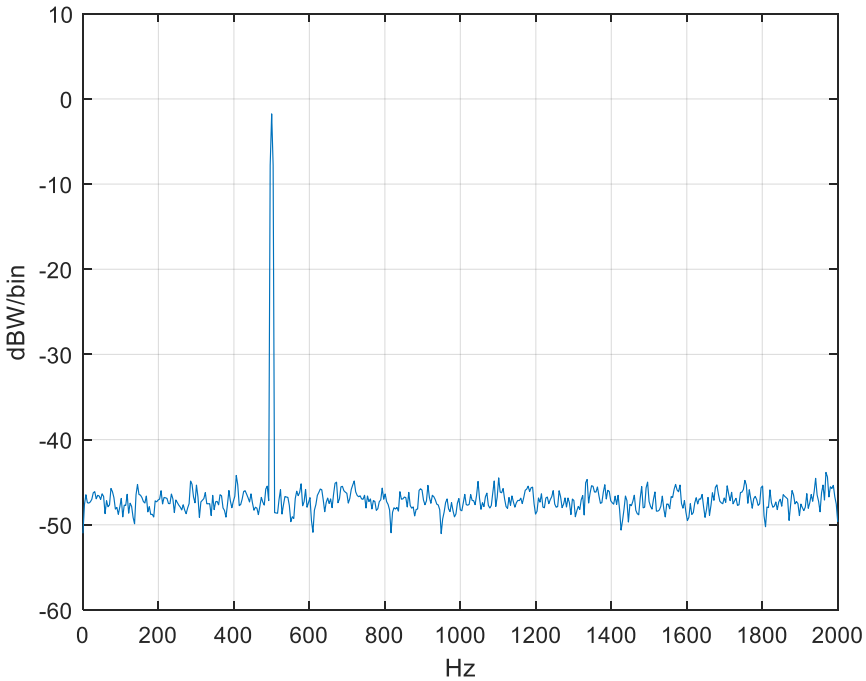


Figure 4. Spectrum with N_{avg} = 15 and noverlap= nfft/2= 512.

Example 4. Calculation of C/N_0

Reducing the noise variance by DFT averaging makes calculation of the noise level and thus carrier-to-noise ratio (C/N) more accurate. Let's calculate C/N_0 for the signal of Example 3, where N_0 is the noise power in a 1 Hz bandwidth.

First, we'll find the power of the sinewave (the "C" in C/N_0) by performing a peak search on the dBW/bin spectrum, then adding the spectral components near the peak to get the total power. This is necessary because windowing has spread the power over several frequency samples.

```
y= max(PdB_bin);                % peak of spectrum
k_peak= find(PdB_bin==y);       % index of peak
Psum= fs/nfft * sum(pxx(k_peak-2:k_peak+2));  %W sum power of 5 samples
C= 10*log10(Psum)               % dBW carrier power
```

The peak power of 0 dBW is shown by the marker in the left plot of Figure 4. Now we'll find N_0 from the dBW/Hz spectrum. Choosing f = 800 Hz, we have:

```
PdB_Hz= 10*log10(pxx);      % dBW/Hz
f1= 800;                    % Hz
k1= round(f1/fs *nfft);     % index of f1
No= PdB_Hz(k1)              % dBW/Hz Noise density at f1
```

The N_0 marker is shown in the right plot of Figure 4. Figure 5 combines the dBW/bin and dBW/Hz plots in one graph. C/N_0 is just:

```
CtoNo= C - No;              % dB (1 Hz)
```

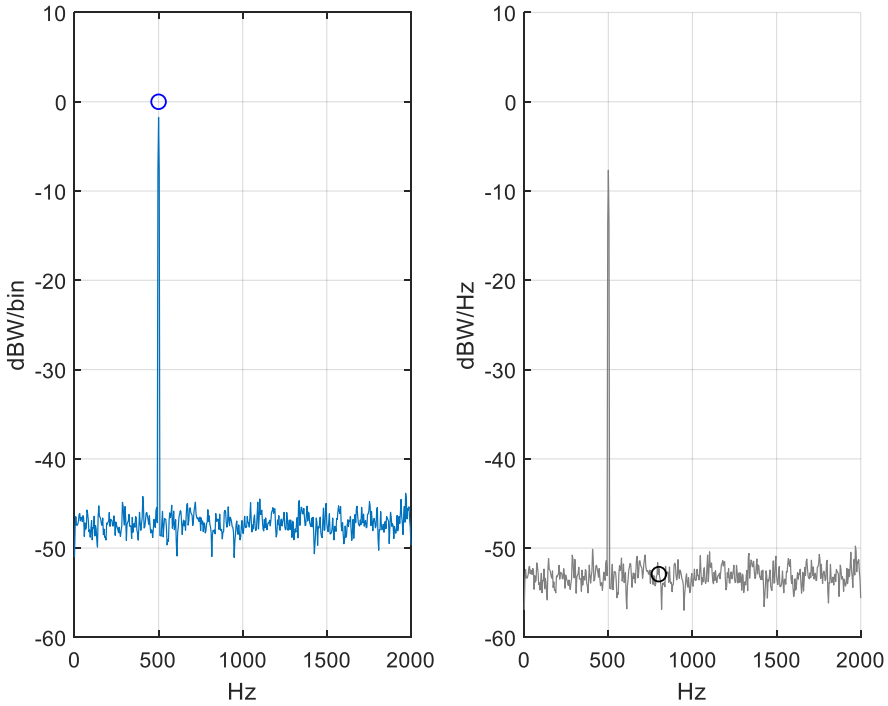


Figure 5. Left: Spectrum in dBW/bin with marker showing the power of the sinewave. Right: Spectrum in dBW/Hz with marker at 800 Hz showing N_0.

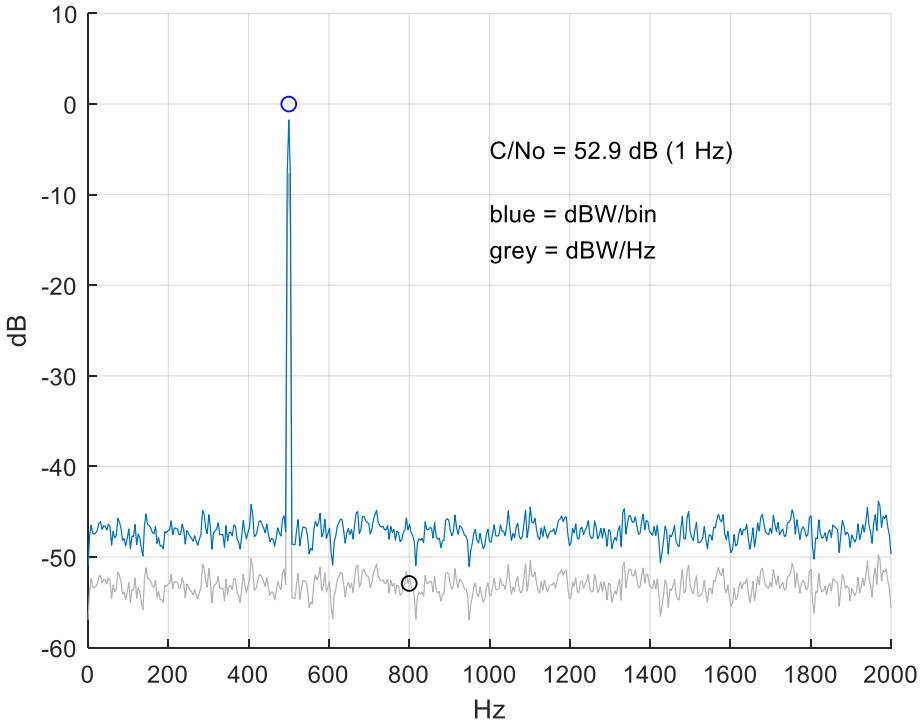


Figure 6. Spectrum in dBW/bin (blue) and dBW/Hz (grey).

Appendix Matlab Code to compute spectra without using pwelch

These examples use the Malab fft function to compute spectra. See my earlier post [7] for derivations of the formulas for power/bin and normalized window function.

Demonstrate Spectrum Computation/ No Averaging

```
%spectrum_demo.m    1/3/19  Neil Robertson

% Use FFT to find spectrum of sine + noise in units of dBW/bin and dBW/Hz.
fs= 4000;               % Hz  sample frequency
Ts= 1/fs;
f0= 500;                % Hz  sine frequency
A= sqrt(2);             % V sine amplitude for P= 1 W into 1 ohm.

N= 1024;                % number of time samples
nfft= N;                % number of frequency samples

n= 0:N-1;               % time index

x= A*sin(2*pi*f0*n*Ts) + .1*randn(1,N);   % 1 W sinewave + noise

w= hanning(N);                  % window function
window= w.*sqrt(N/sum(w.^2));   % normalize window for P= 1

xw= x.*window';                 % apply window to x

X= fft(xw,nfft);                % DFT
X= X(1:nfft/2);                 % retain samples from 0 to fs/2

magsq= real(X).^2 + imag(X).^2; % DFT magnitude squared
P_bin= 2/nfft.^2 * magsq;       % W/bin power spectrum into 1 ohm
P_Hz= P_bin*nfft/fs;            % W/Hz power spectrum

PdB_bin= 10*log10(P_bin);       % dBW/bin
PdB_Hz= 10*log10(P_Hz);         % dBW/Hz

k= 0:nfft/2-1;                  % frequency index
f= k*fs/nfft;                   % Hz  frequency vector

plot(f,PdB_bin),grid
axis([0 fs/2 -60 10])
xlabel('Hz'),ylabel('dBW/bin')
title('Spectrum with amplitude units of dBW/bin')
figure

y= max(PdB_Hz);
plot(f,PdB_Hz),grid
axis([0 fs/2 y-60 y+10])
xlabel('Hz'),ylabel('dBW/Hz')
title('Spectrum with amplitude units of dBW/Hz')
```

Demonstrate Spectrum Computation with Averaging

Suppose we have segmented a signal x into four segments of length nfft, that could be overlapped. If we window each segment and take its DFT, we have four DFT's as shown in Table A.1. Here, we have defined M= nfft and we have numbered the FFT bins from 1 to M (instead of 0 to M - 1).

Table A.1 DFT's of four segments of the signal x.

BIN	DFT 1	DFT 2	DFT 3	DFT 4
k= 1	X_{11}	X_{12}	X_{13}	X_{14}
k= 2	X_{21}	X_{22}	X_{23}	X_{24}
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
k= M= nfft	X_{M1}	X_{M2}	X_{M3}	X_{M4}

Now we form the magnitude-squared of each bin of the four DFT's, as shown in Table A.2 If we then sum the magnitude-square values of each bin over the four DFT's and divide by 4, we obtain the average magnitude-squared value, as shown in the right column. Finally, we scale the magnitude-squared value to obtain the averaged power per bin [7] or averaged power per Hz:

$$P_{bin}(k) = 2/M^2 * \text{sum}(|X_k|^2)/4 \quad \text{W/bin} \qquad \text{(A.1)}$$

$$P_{Hz}(k)= P_{bin}(k)*M/f_s \quad \text{W/Hz} \qquad \text{(A.2)}$$

Or

$$P_{Hz}(k)= 2/(M*f_s) * \text{sum}(|X_k|^2)/4 \quad \text{W/Hz} \qquad \text{(A.3)}$$

$P_{Hz}(k)$ is the Welch power spectral density estimate.

Table A.2 Magnitude-square values of Four DFT's, and their average (right column).

BIN	DFT 1	DFT 2	DFT 3	DFT 4	DFT mag sq. average
k= 1	$\|X_{11}\|^2$	$\|X_{12}\|^2$	$\|X_{13}\|^2$	$\|X_{14}\|^2$	$\text{sum}(\|X_1\|^2)/4$
k= 2	$\|X_{21}\|^2$	$\|X_{22}\|^2$	$\|X_{23}\|^2$	$\|X_{24}\|^2$	$\text{sum}(\|X_2\|^2)/4$
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
k= M= nfft	$\|X_{M1}\|^2$	$\|X_{M2}\|^2$	$\|X_{M3}\|^2$	$\|X_{M4}\|^2$	$\text{sum}(\|X_M\|^2)/4$

The following example m-file uses N_{avg} = 8, summing 8 values of each $|X(k)|^2$ and dividing by 8. For simplicity, noverlap = 0 samples. Another option for averaging, useful for the case where DFT's are taken continuously, is exponential averaging, which basically runs the $|X(k)|^2$ through a first-order IIR filter [8].

```
%spectrum_demo_avg     1/3/19 Neil Robertson
% Perform DFT averaging on signal = sine + noise
% noverlap = 0
% Given signal x(1:8192),
%
%            |x(1)     x(1025) . . . . x(7169)|
%            |x(2)     x(1026) . . . . x(7170)|
%  xMatrix = | .          .                .   |
%            | .          .                .   |
%            |x(1024)   x(2048) . . . . x(8192)|
%
%
fs= 4000;                   % Hz  sample frequency
Ts= 1/fs;
f0= 500;                    % Hz  sine frequency
A= sqrt(2);                 % V  sine amplitude for P= 1 W into 1 ohm

nfft= 1024;                 % number of frequency samples
Navg= 8;                    % number of DFT's to be averaged
N= nfft*Navg;               % number of time samples

n= 0:N-1;                   % time index

x= A*sin(2*pi*f0*n*Ts) + .1*randn(1,N);     % 1 W sinewave + noise

w= hanning(nfft);                   % window function
window= w.*sqrt(nfft/sum(w.^2));    % normalize window for P= 1 W

% Create matrix xMatrix with Navg columns,
% each column a segment of x of length nfft
xMatrix= reshape(x,nfft,Navg);
magsq_sum= zeros(nfft/2);

for i= 1:Navg
    x_column= xMatrix(:,i);
    xw= x_column.*window;           % apply window of length nfft

    X= fft(xw);                     % DFT
    X= X(1:nfft/2);                 % retain samples from 0 to fs/2

    magsq= real(X).^2 + imag(X).^2;       % DFT magnitude squared
    magsq_sum= magsq_sum + magsq;         % sum of DFT mag squared
end

mag_sq_avg= magsq_sum/Navg;          % average of DFT mag squared

P_bin= 2/nfft.^2 *mag_sq_avg;        % W/bin  power spectrum
P_Hz= P_bin*nfft/fs;                 % W/Hz   power spectrum
```

```
PdB_bin= 10*log10(P_bin);              % dBW/bin
PdB_Hz= 10*log10(P_Hz);                % dBW/Hz

k= 0:nfft/2 -1;                        % frequency index
f= k*fs/nfft;                          % Hz  frequency vector

plot(f,PdB_bin),grid
axis([0 fs/2 -60 10])
xlabel('Hz'),ylabel('dBW/bin')
title('Averaged Spectrum with amplitude units of dBW/bin')
figure

plot(f,PdB_Hz),grid
axis([0 fs/2 -60 10])
xlabel('Hz'),ylabel('dBW/Hz')
title('Averaged Spectrum with amplitude units of dBW/Hz')
```

References

1. Robertson, Neil, "Evaluate Window Functions for the Discrete Fourier Transform" https://www.dsprelated.com/showarticle/1211.php
2. Mathworks website https://www.mathworks.com/help/signal/ref/pwelch.html
3. Oppenheim, Alan V. and Shafer, Ronald W., Discrete-Time Signal Processing, Prentice Hall, 1989, pp. 737—742.
4. Welch, Peter D., "The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms", IEEE Trans. Audio and Electroacoustics, vol AU-15, pp. 70 – 73, June 1967. https://www.utd.edu/~cpb021000/EE%204361/Great%20DSP%20Papers/Welchs%20Periodogram.pdf
5. Lyons, Richard G., Understanding Digital Signal Processing, 2nd Ed., Prentice Hall, 2004, section 3.8.
6. Oppenheim and Shafer, p. 738.
7. Robertson, Neil, "The Power Spectrum", https://www.dsprelated.com/showarticle/1004.php
8. Lyons, section 11.5.

Neil Robertson January, 2019